

Индивидуальный проект. Этап 1

Размещение на Github pages заготовки для персонального сайта

Игнатенко Денис Беньяминович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение индивидуального проекта	8
4.1	Установка программного обеспечения	8
4.2	Создание репозитория на GitHub	9
4.3	Клонирование репозитория	10
4.4	Установка зависимостей проекта	11
4.5	Локальный запуск сайта	11
4.6	Настройка baseUrl	12
4.7	Настройка GitHub Pages	13
4.8	Публикация сайта	14
4.9	Проверка результата	15
5	Основные команды	16
6	Выводы	17
	Список литературы	18

Список иллюстраций

4.1	Установка git, go lang, nodejs, npm	8
4.2	Проверка версий установленного ПО	9
4.3	Установка Hugo Extended	9
4.4	Создание репозитория по шаблону	10
4.5	Созданный репозиторий на GitHub	10
4.6	Клонирование репозитория	11
4.7	Установка прт-зависимостей	11
4.8	Запуск локального сервера Hugo	12
4.9	Локальный сайт в браузере	12
4.10	Настройка baseURL в config.yaml	13
4.11	Настройка GitHub Pages	14
4.12	Коммит и пуш изменений	14
4.13	Успешный деплой в GitHub Actions	15
4.14	Сайт на публичном адресе	15

Список таблиц

3.1	Используемые технологии и инструменты {#tbl:tools}	7
5.1	Основные команды {#tbl:commands}	16

1 Цель работы

Разместить на Github pages заготовку для персонального сайта научного работника: установить необходимое программное обеспечение, скачать шаблон темы сайта, разместить его на хостинге git и настроить параметры для корректного отображения сайта.

2 Задание

- Установить необходимое программное обеспечение (Git, Go, Node.js, Hugo Extended)
- Скачать шаблон темы сайта Hugo Academic
- Создать репозиторий на GitHub и разместить шаблон
- Установить параметр baseURL для корректной работы сайта
- Разместить заготовку сайта на Github pages

3 Теоретическое введение

Hugo — генератор статических сайтов, написанный на языке Go. Он позволяет создавать быстрые и безопасные веб-сайты без необходимости использования баз данных или серверных скриптов. Hugo использует шаблоны и Markdown-файлы для генерации HTML-страниц.

Hugo Academic Theme (ныне HugoBlox) — популярный шаблон для создания персональных академических сайтов. Он предоставляет готовую структуру для размещения информации о публикациях, проектах, курсах и биографии исследователя.

GitHub Pages — бесплатный хостинг статических сайтов от GitHub. Позволяет публиковать сайты напрямую из репозитория, поддерживает автоматическую сборку через GitHub Actions.

Основные технологии и инструменты представлены в таблице ??.

Таблица 3.1: Используемые технологии и инструменты {#tbl:tools}

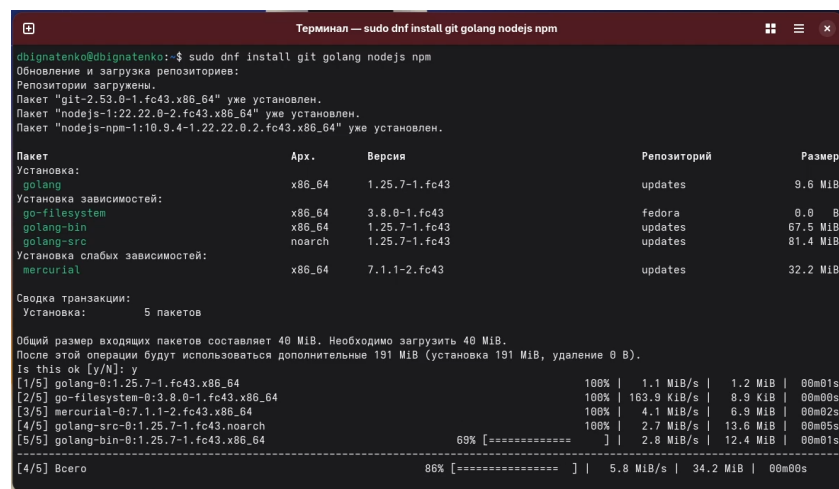
Инструмент	Описание
Hugo	Генератор статических сайтов
Go	Язык программирования (необходим для Hugo)
Node.js/npm	Среда выполнения JavaScript (для зависимостей темы)
Git	Система контроля версий
GitHub Pages	Хостинг статических сайтов
GitHub Actions	CI/CD для автоматической сборки и деплоя

4 Выполнение индивидуального проекта

4.1 Установка программного обеспечения

Устанавливаю необходимые пакеты через менеджер пакетов dnf: Git, Go (golang) и Node.js с npm (рис. 4.1).

```
sudo dnf install git golang nodejs npm
```



```
Терминал — sudo dnf install git golang nodejs npm
dbignatenko@dbignatenko:~$ sudo dnf install git golang nodejs npm
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет "git-2.53.0-1.fc43.x86_64" уже установлен.
Пакет "nodejs-1:22.22.0-2.fc43.x86_64" уже установлен.
Пакет "nodejs-npm-1:10.9.4-1.22.22.0.2.fc43.x86_64" уже установлен.

Пакет      Арх.      Версия      Репозиторий      Размер
Установка:
golang      x86_64      1.25.7-1.fc43      updates           9.6 MiB
Установка зависимостей:
go-filestore x86_64      3.8.0-1.fc43      fedora            0.0 B
golang-bin   x86_64      1.25.7-1.fc43      updates          67.5 MiB
golang-src   noarch      1.25.7-1.fc43      updates          81.4 MiB
Установка слабых зависимостей:
mercurial    x86_64      7.1.1-2.fc43      updates          32.2 MiB

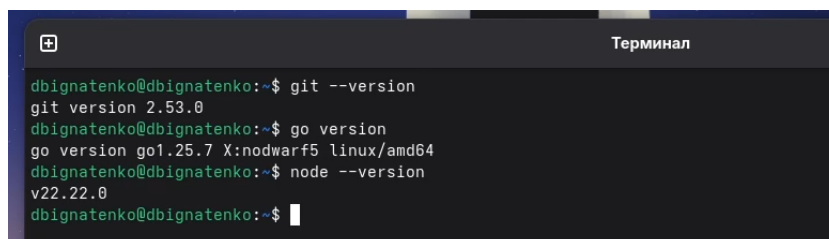
Сводка транзакции:
Установка:      5 пакетов

Общий размер входящих пакетов составляет 40 MiB. Необходимо загрузить 40 MiB.
После этой операции будут использоваться дополнительные 191 MiB (установка 191 MiB, удаление 0 B).
Is this ok [y/N]: y
[1/5] golang-0:1.25.7-1.fc43.x86_64      100% | 1.1 MiB/s | 1.2 MiB | 00m01s
[2/5] go-filestore-0:3.8.0-1.fc43.x86_64 100% | 163.9 KiB/s | 8.9 KiB | 00m00s
[3/5] mercurial-0:7.1.1-2.fc43.x86_64     100% | 4.1 MiB/s | 6.9 MiB | 00m02s
[4/5] golang-src-0:1.25.7-1.fc43.noarch   100% | 2.7 MiB/s | 13.6 MiB | 00m05s
[5/5] golang-bin-0:1.25.7-1.fc43.x86_64   69% [===== ] | 2.8 MiB/s | 12.4 MiB | 00m01s
[4/5] Всего                               86% [===== ] | 5.8 MiB/s | 34.2 MiB | 00m00s
```

Рис. 4.1: Установка git, golang, nodejs, npm

Проверяю корректность установки, выводя версии установленных программ (рис. 4.2).


```
git --version
go version
node --version
```

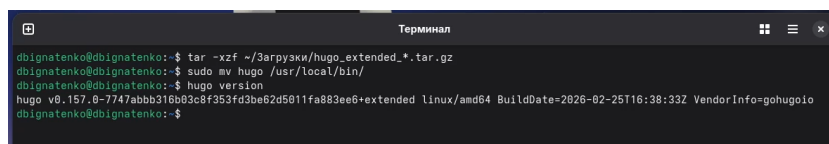
A terminal window titled "Терминал" showing the following commands and their outputs:

```
dbignatenko@dbignatenko:~$ git --version
git version 2.53.0
dbignatenko@dbignatenko:~$ go version
go version go1.25.7 X:nodwarf5 linux/amd64
dbignatenko@dbignatenko:~$ node --version
v22.22.0
dbignatenko@dbignatenko:~$
```

Рис. 4.2: Проверка версий установленного ПО

Устанавливаю Hugo Extended вручную. Скачиваю архив с официального репозитория GitHub, распаковываю его и перемещаю бинарный файл в `/usr/local/bin/`. Проверяю версию — важно, чтобы была версия Extended (рис. 4.3).

```
tar -xzf ~/Загрузки/hugo_extended_*.tar.gz
sudo mv hugo /usr/local/bin/
hugo version
```

A terminal window titled "Терминал" showing the following commands and their outputs:

```
dbignatenko@dbignatenko:~$ tar -xzf ~/Загрузки/hugo_extended_*.tar.gz
dbignatenko@dbignatenko:~$ sudo mv hugo /usr/local/bin/
dbignatenko@dbignatenko:~$ hugo version
hugo v0.157.0-7747abbb316b03c8f353fd3be62d5011fa883ee6+extended Linux/amd64 BuildDate=2026-02-25T16:38:33Z VendorInfo=gohugoio
dbignatenko@dbignatenko:~$
```

Рис. 4.3: Установка Hugo Extended

4.2 Создание репозитория на GitHub

Перехожу на страницу шаблона `HugoBlox/hugo-theme-academic-cv` на GitHub и нажимаю «Use this template». Создаю новый репозиторий с именем `den4ik-dev.github.io` — это важно, так как имя репозитория должно соответствовать формату `USERNAME.github.io` для корректной работы GitHub Pages (рис. 4.4).

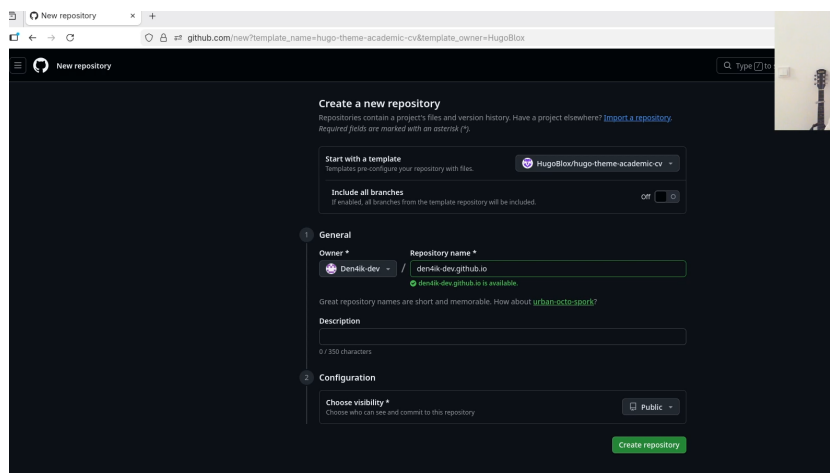


Рис. 4.4: Создание репозитория по шаблону

Репозиторий успешно создан на основе шаблона. Он содержит все необходимые файлы конфигурации и структуру для Hugo-сайта (рис. 4.5).

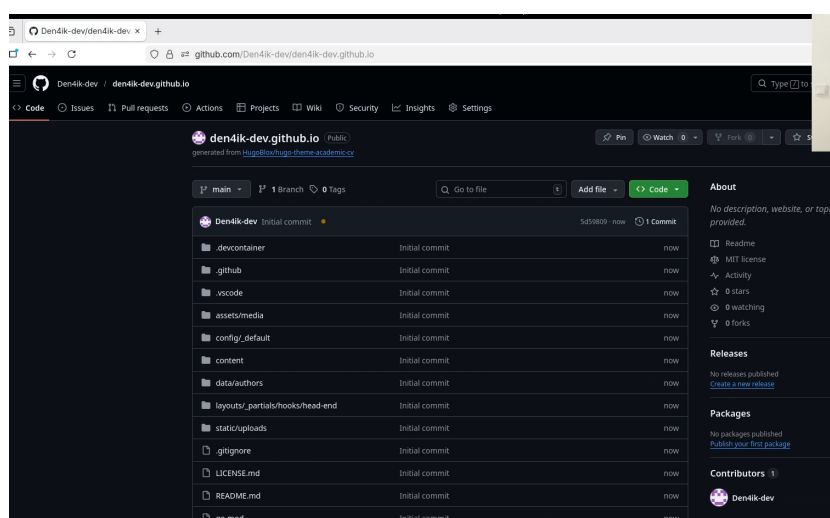
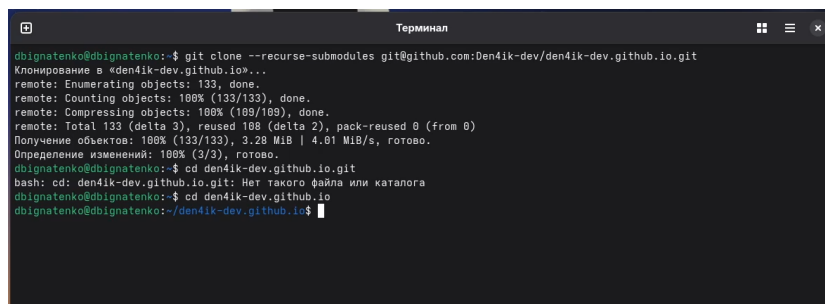


Рис. 4.5: Созданный репозиторий на GitHub

4.3 Клонирование репозитория

Клонирую созданный репозиторий на локальную машину с флагом `--recursive-submodules` для загрузки всех подмодулей темы (рис. 4.6).

```
git clone --recurse-submodules git@github.com:Den4ik-dev/den4ik-dev.github.io.git
cd den4ik-dev.github.io
```



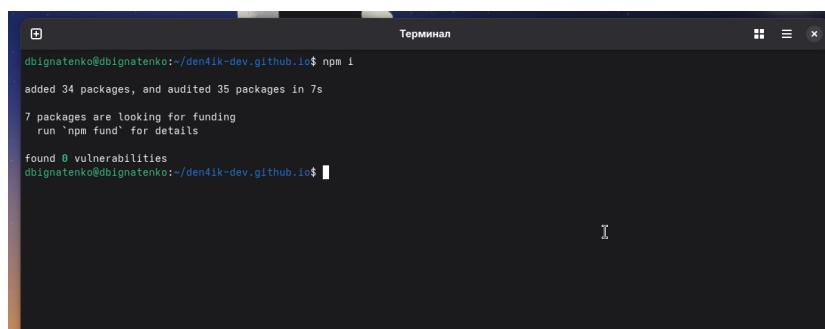
```
Терминал
dbignatenko@dbignatenko:~$ git clone --recurse-submodules git@github.com:Den4ik-dev/den4ik-dev.github.io.git
Клонирование в «den4ik-dev.github.io»...
remote: Enumerating objects: 133, done.
remote: Counting objects: 100% (133/133), done.
remote: Compressing objects: 100% (109/109), done.
remote: Total 133 (delta 3), reused 108 (delta 2), pack-reused 0 (from 0)
Получение объектов: 100% (133/133), 3.28 MiB | 4.01 MiB/s, готово.
Определение изменений: 100% (3/3), готово.
dbignatenko@dbignatenko:~$ cd den4ik-dev.github.io.git
bash: cd: den4ik-dev.github.io.git: Нет такого файла или каталога
dbignatenko@dbignatenko:~$ cd den4ik-dev.github.io
dbignatenko@dbignatenko:~/den4ik-dev.github.io$
```

Рис. 4.6: Клонирование репозитория

4.4 Установка зависимостей проекта

Устанавливаю npm-зависимости проекта командой `npm i`. Это установит все необходимые JavaScript-пакеты, включая TailwindCSS и другие инструменты для сборки (рис. 4.7).

```
npm i
```



```
Терминал
dbignatenko@dbignatenko:~/den4ik-dev.github.io$ npm i
added 34 packages, and audited 35 packages in 7s

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
dbignatenko@dbignatenko:~/den4ik-dev.github.io$
```

Рис. 4.7: Установка npm-зависимостей

4.5 Локальный запуск сайта

Запускаю локальный сервер разработки командой `hugo server`. Hugo компилирует сайт и запускает веб-сервер на `localhost:1313` (рис. 4.8).

hugo server

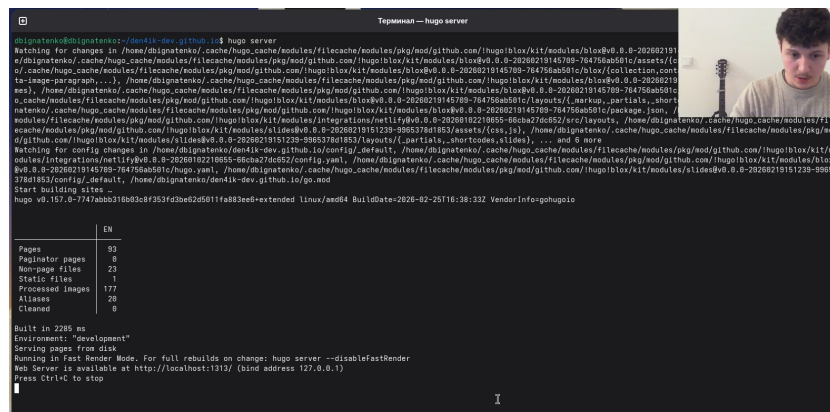


Рис. 4.8: Запуск локального сервера Hugo

Открываю браузер и перехожу по адресу `http://localhost:1313`. Сайт загружается корректно — отображается шаблон с демонстрационным контентом (рис. 4.9).

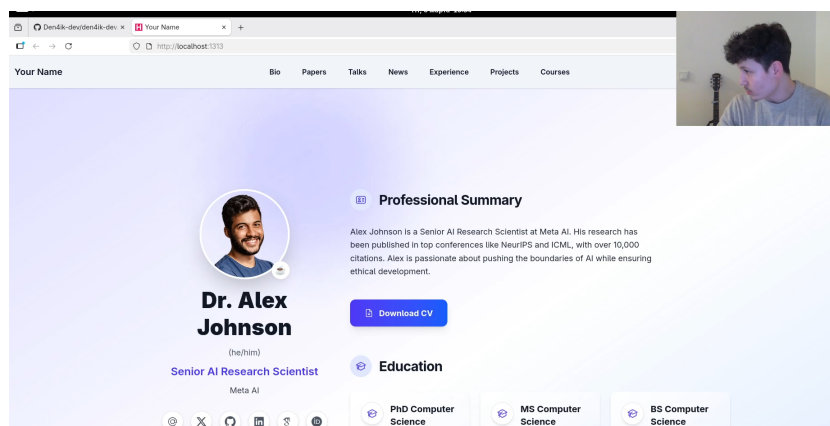


Рис. 4.9: Локальный сайт в браузере

4.6 Настройка baseURL

Открываю файл конфигурации `config/_default/config.yaml` и устанавливаю корректный `baseURL`, соответствующий адресу моего сайта на GitHub Pages (рис. 4.10).

```
nano config/_default/config.yaml
```

Устанавливаю значение:

```
baseURL: 'https://den4ik-dev.github.io/'
```

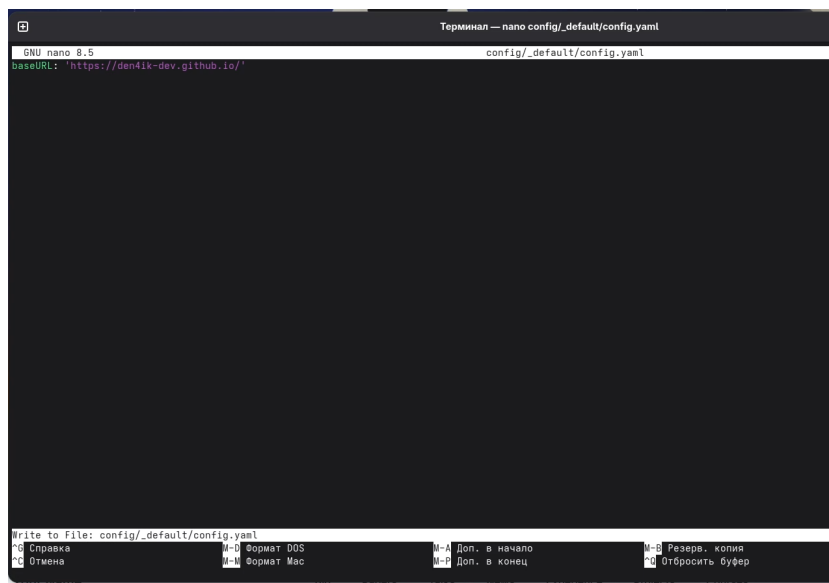


Рис. 4.10: Настройка baseURL в config.yaml

4.7 Настройка GitHub Pages

В настройках репозитория на GitHub перехожу в раздел Settings → Pages. В качестве источника (Source) выбираю GitHub Actions — это позволит автоматически собирать и публиковать сайт при каждом пуше в репозиторий (рис. 4.11).

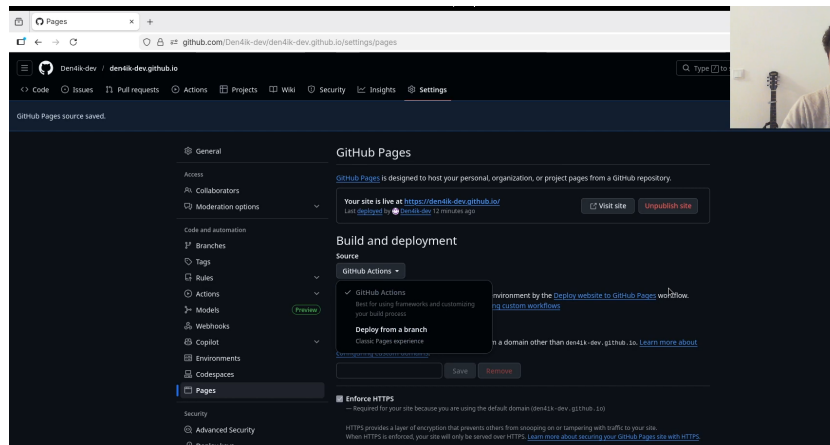


Рис. 4.11: Настройка GitHub Pages

4.8 Публикация сайта

Фиксирую все изменения и отправляю их на GitHub (рис. 4.12).

```
git add .
git commit -m "initial site setup"
git push
```

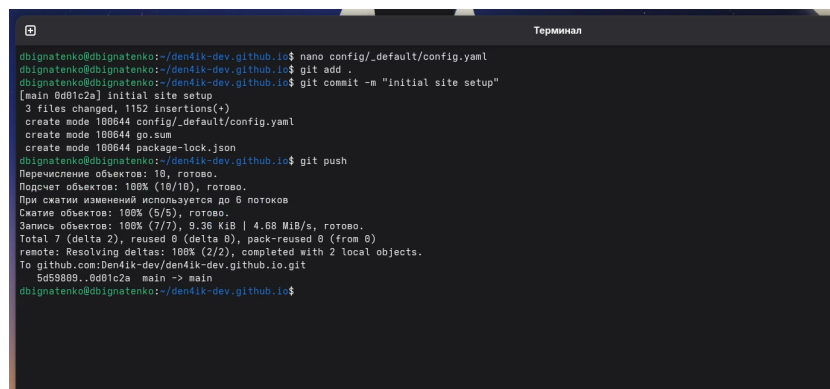


Рис. 4.12: Коммит и пуш изменений

Перехожу во вкладку Actions на GitHub и ожидаю успешного завершения workflow. Все три этапа (config, build, deploy) выполнены успешно (рис. 4.13).

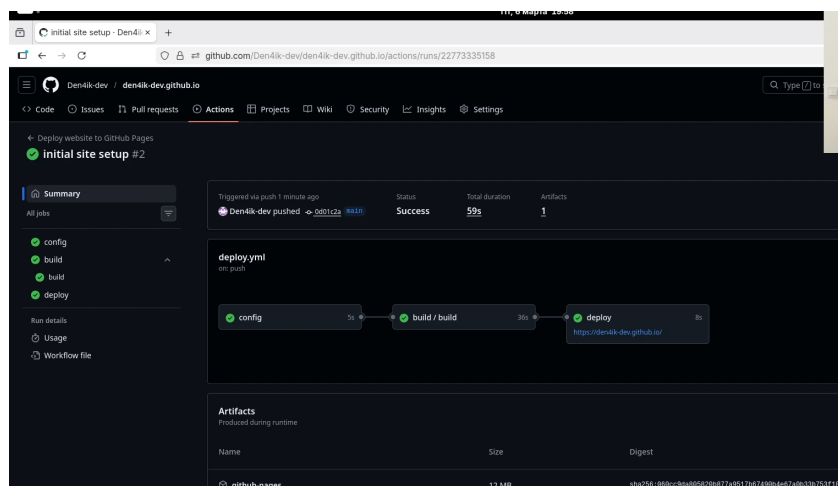


Рис. 4.13: Успешный деплой в GitHub Actions

4.9 Проверка результата

Открываю сайт по публичному адресу <https://den4ik-dev.github.io/>. Сайт отображается корректно — загружаются стили, шрифты и демонстрационный контент шаблона (рис. 4.14).

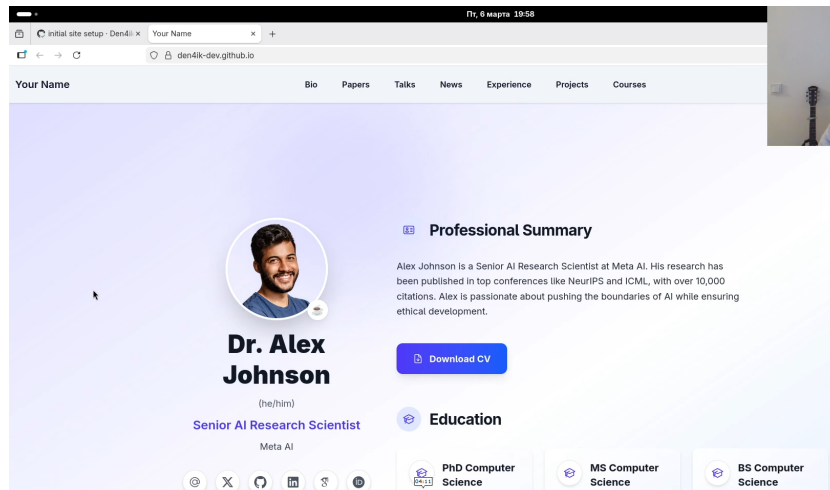


Рис. 4.14: Сайт на публичном адресе

5 Основные команды

Основные команды, использованные при выполнении проекта, представлены в таблице ??.

Таблица 5.1: Основные команды {#tbl:commands}

Команда	Описание
<code>sudo dnf install git golang</code>	Установка необходимого ПО
<code>nodejs npm</code>	
<code>hugo version</code>	Проверка версии Hugo
<code>git clone --recurse-submodules</code> <code><url></code>	Клонирование репозитория с подмодулями
<code>npm i</code>	Установка npm-зависимостей
<code>hugo server</code>	Запуск локального сервера разработки
<code>git add .</code>	Добавление всех изменений в индекс
<code>git commit -m "message"</code>	Создание коммита
<code>git push</code>	Отправка изменений на GitHub

6 Выводы

В ходе выполнения первого этапа индивидуального проекта установил необходимое программное обеспечение (Git, Go, Node.js, Hugo Extended), создал репозиторий на GitHub на основе шаблона Hugo Academic Theme, настроил baseURL и GitHub Pages для публикации сайта. Заготовка персонального сайта научного работника успешно размещена и доступна по адресу <https://den4ik-dev.github.io/>.

Список литературы